DEVSYNC

Secure AI Project Collaborator

Project Description and Software Requirements Specification

Connect. Collaborate. Build.

1. Project Overview

1.1 Project Name

DevSync — Secure AI Project Collaborator.

1.2 Brief Introduction

DevSync is an Android mobile application intended to help developers—particularly students and junior developers—discover open-source or collaborative projects that align with their demonstrated skills. It uses profile information and available public GitHub activity to build a skill profile, recommend relevant projects, explain the reasons behind each recommendation, and identify skill gaps. It also supports project owners in defining project needs, finding contributors, handling collaboration requests, and coordinating project tasks.

The product brings project discovery and essential collaboration activities into one mobile experience. Its guiding statement is “Connect. Collaborate. Build.”

1.3 Main Purpose and Objectives

- Recommend projects using AI-assisted matching and provide a human-readable explanation for each recommendation.
- Show relevant skill gaps and concise learning suggestions so users can understand what they may need to develop.
- Support secure account access, developer profiles, GitHub data import, project creation, collaboration requests, and task coordination.
- Help project owners describe the required tracks and identify unfilled team positions.
- Collect user feedback on AI outputs and compare prompt versions to evaluate recommendation quality.
- Deliver Android release variants (Free and Pro) with code obfuscation and technical documentation.

1.4 Project Context and Delivery

Attribute	Source-aligned description
Project type	DEPI graduation project
Team	Six Flutter developers; project manager / Flutter lead: Mohraeel
Platform and package	Android; Android App Bundle (AAB) and APK for testing
Implementation stack	Flutter, BLoC/Cubit, Firebase, GitHub REST API, Gemini through Firebase AI Logic
Planning period	5 October–4 December 2026 (approximately nine weeks)
Delivery approach	MoSCoW prioritization; Must features define the MVP

The MVP is expected to demonstrate the end-to-end path: profile setup → project matching → collaboration request → team membership → task work. Optional and lower-priority features remain conditional on available time.

2. Problem Statement and Rationale

2.1 Problem to Be Solved

Developers who want to contribute to projects often struggle to identify opportunities that match their actual experience and current level. Project owners face the inverse problem: they need contributors with appropriate skills and may overlook important roles in the team. Existing discovery approaches described in the project materials rely heavily on manual searching and do not adequately explain why a project or developer is a suitable match.

2.2 Current User Challenges

- Beginners and students may not know which open-source or collaborative projects are realistic for their current skill level.

- Self-reported skills may not reflect practical experience, while useful evidence may be distributed across GitHub repositories, languages, and activity.
- Users may see a match result without understanding which skills match, which are complementary, or which are missing.
- Project owners may have difficulty identifying missing tracks such as backend or QA and locating suitable people for them.
- Project discovery, collaboration requests, team coordination, and task tracking can be fragmented across separate tools.
- Manual project descriptions and role definitions can make it harder to communicate a project's scope and team needs.

2.3 Why the Problem Matters

A poor match can discourage early-career developers, create unrealistic expectations, and leave project teams without necessary capabilities. Explainable recommendations and visible skill gaps can make participation more informed, while integrated collaboration workflows can reduce friction between discovering an opportunity and contributing to it.

2.4 Proposed Solution

DevSync combines profile-based discovery, GitHub-informed skill evidence, explainable matching, skill-gap guidance, project and track management, collaboration requests and invitations, and task coordination. AI-generated content remains reviewable: detected skills are presented for user approval, project setup suggestions can be edited, and an AI-drafted collaboration pitch is reviewed by the applicant before sending.

2.5 Expected Benefits

- More relevant project discovery for developers at different experience levels.
- Clearer understanding of recommendation reasons, matched skills, complementary skills, and missing skills.
- Practical direction for learning skills needed by a target project.
- Improved visibility of project capacity and missing team tracks.
- A more continuous workflow from discovery through team participation and task management.
- Evidence for evaluating and improving prompt quality through structured user feedback.

3. Intended Users and Roles

The same person may act as a project owner in one project and a member/applicant in another. The role is therefore contextual to each project rather than a permanent account-wide role.

User group / role	Needs and goals	How DevSync supports the group
Developer / contributor	Find projects that fit current skills, experience, interests, and level; understand gaps; join a team.	Profile, GitHub-derived information, discovery filters, AI recommendations, match explanation, skill-gap tips, join requests, and tasks.
Beginner / student	Find approachable learning opportunities and understand what to learn next.	Project purpose and difficulty, explainable match, missing-skill list, and short learning tips.
Project owner	Describe a project, define required tracks and capacity, find contributors, and manage participation.	Project creation, editable AI setup/role suggestions where available, track slots, missing-role indication, incoming requests, invitations, and project tasks.
Guest	Explore public projects and selected public developer information before creating an account.	Read-only project browsing; actions requiring an account lead to authentication and return to the intended destination. Guest access to public profile fields needs confirmation.
Team member	Understand project work, view team information, and complete assigned tasks.	Team workspace, member list, task board, task details, and (if included in the selected scope) team communication.

3.1 Project-Level Roles

- Project Owner: creates and publishes the project, defines tracks, reviews and decides join requests, and may invite developers. The owner is counted as a team member and fills one slot in the ownerTrack.
- Member / Applicant: discovers a project, may submit one join request for that project, and becomes a team member after acceptance.
- Invitee: receives an owner-created invitation and may accept or decline it, subject to track capacity.
- Guest: can read projects; non-read actions require sign-in. The exact public developer profile fields available to Guests are Needs Confirmation.

4. Application Description and High-Level Operation

4.1 Application Components

Component	Responsibilities / interaction
Flutter mobile client	Screens, navigation, forms, validation, state management through BLoC/Cubit, and user interactions.
Firebase Authentication	Google and Email sign-in according to the current requirements baseline.
Cloud Firestore	Profiles, projects, requests, invitations, teams, tasks, matches, feedback, prompts, usage counters, and in-app notifications.
GitHub public REST API	The app reads public repository information using the user's GitHub username; no authenticated token is used in the Spark-only design.
Gemini via Firebase AI Logic	AI-assisted matching/explanations, skill-gap suggestions, project setup suggestions, and collaboration pitch drafting as described in the project materials.
External file host	Profile photo and CV files are stored externally; Firestore stores their URLs. Firebase Storage is not used.
Android local notification mechanism	Can show a local notification while the app is running; the Spark-only design does not provide push delivery when the app is closed.

4.2 High-Level Workflow

1. The user opens DevSync and either signs in or continues as a Guest for permitted browsing.
2. A signed-in developer completes a profile with name, track, level, and at least one programming language. Other profile information is optional or supplementary.
3. The user may provide a GitHub username. The app reads public repositories, languages, topics, and activity, subject to public API rate limits and a roughly one-hour re-sync throttle.
4. Detected skills are shown separately from confirmed/manual skills. The user reviews detected skills and decides whether to add them.
5. The discovery experience presents projects through browsing, filters, and matching. The matching service considers skill overlap, open track, level versus difficulty, interests, and purpose.
6. For a recommendation, the app shows an explanation and matched/missing or complementary skills rather than presenting a score alone. Short learning tips may be shown for missing skills.
7. The developer may save a project, express interest, or submit a join request. These are separate actions. A generated pitch is reviewed and edited by the user before submission.
8. The owner reviews incoming requests and accepts or rejects them. Acceptance adds the applicant to the team and fills the selected track slot, subject to capacity.
9. An owner may create a project manually or use AI assistance where available. AI-generated project details and role suggestions are editable; publication occurs only after the owner confirms.
10. Team members use project workspace and task functions to create, assign, and update tasks. Task changes are reflected through Firestore data and in-app notification records.
11. Users can rate AI outputs. Feedback is stored with the prompt version so the team can compare helpfulness and clarity across prompt versions.

4.3 Navigation and User Experience

The signed-in application design specifies five persistent bottom-navigation tabs: Discover, Community, My Projects, Messages, and Profile. Notifications are accessed through a bell icon in the top app bar rather than a separate bottom tab. The Guest experience is limited to permitted browsing; when a Guest attempts an account-required action, authentication should return the user to the original screen or destination.

The supplied design system specifies Inter typography, a blue-led visual identity, light and dark themes, an 8-point spacing grid, 48×48 px minimum interactive hit areas, 12 px button/input radii, 16 px card radii, and outlined icons except for active navigation states.

4.4 AI Design Principles

- Explainability: a match score must be accompanied by a reason and skill context.
- Evidence separation: AI-detected skills are not presented as verified facts until the user accepts them.
- Human review: generated project setup, roles, and collaboration pitch are editable and require user action before publication or sending.
- Graceful degradation: if AI is unavailable or slow, cached or rule-based results should be shown where available, with an explanatory notice.
- Feedback: helpfulness and clarity ratings are associated with the prompt version that produced the output.

5. Scope and Prioritization

Priority	Scope interpretation from the supplied materials
Must / MVP	Authentication; profile and GitHub data; project discovery and filters; AI recommendations, explanations, skill gaps and feedback; project creation; task management; collaboration requests; in-app notification records; Free/Pro flavors; obfuscated Android release.
Should	AI project setup and role generation; missing-role alert; invitations/shared links; workspace and communication items subject to the current schema and time.
Could	Match history, save/favorites, Guest mode enhancements, and Digital Twin (definition not finalized).
Out of scope / excluded in current schema	iOS; payment processing for Pro; Cloud Functions; Firebase Storage; Community and Chatbot are removed from the Firestore schema v1.0 changelog, although earlier planning/design material still mentions Community.

The latest Firestore schema explicitly selects the Firebase Spark plan only. Accordingly, this document treats the no-Cloud-Functions/no-Firebase-Storage architecture as the implementation baseline and records older conflicting statements as Needs Confirmation rather than silently retaining them.

6. Use Cases

The supplied project analysis contains five textual use cases. The following descriptions preserve their original actors, core sequence, and alternative paths while improving grammar and presentation. No replacement use-case diagram has been invented.

UC-01 — Sign In and Complete Profile

Element	Description
Primary actors	Developer
Main flow	The developer signs in through Firebase Authentication, completes the profile setup, optionally connects GitHub, and imports available GitHub information.
Alternative / exception flow	If authentication fails, the system displays an error and allows retry. If GitHub connection/import fails or is skipped, the developer continues using manually entered profile data.

UC-02 — Obtain AI Recommendations

Element	Description
Primary actors	Developer
Main flow	The developer opens Discovery. The app sends the available profile information to the matching service and displays a ranked project list. The developer opens a project to review its match score, explanation, and skill gap, then may submit feedback.
Alternative / exception flow	If AI is slow or unavailable, cached or rule-based results are displayed with a notice. If the profile is incomplete, the app requests completion before matching.

UC-03 — Send and Decide a Collaboration Request

Element	Description
Primary actors	Developer; Project Owner
Main flow	The developer submits a collaboration request. The request is stored and the owner is notified. The owner accepts or rejects it. On acceptance, the developer is added to the team and the relevant parties are notified.
Alternative / exception flow	On rejection, the developer is notified and the request is closed. A duplicate request is prevented.

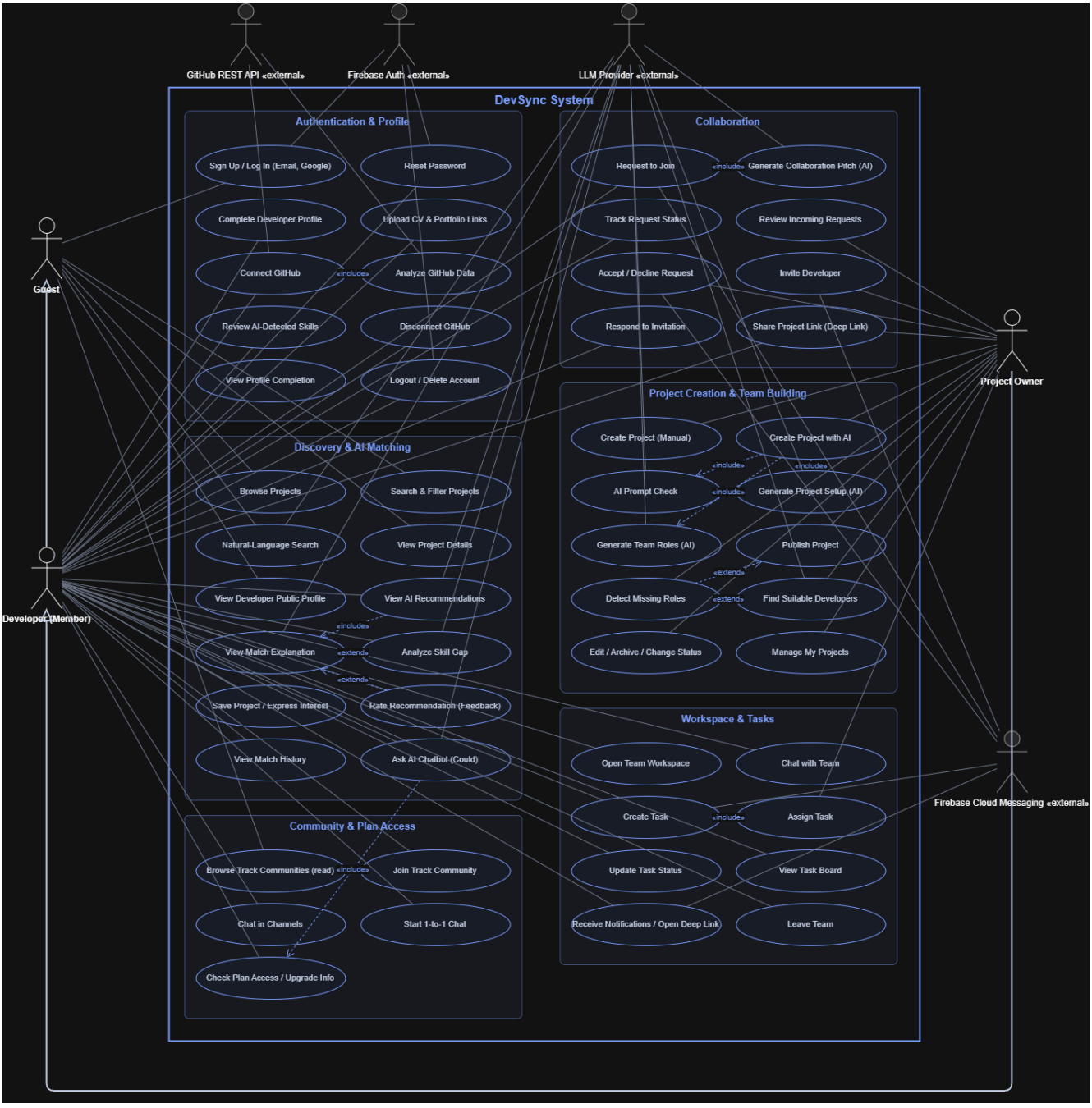
UC-04 — Create a Project and Generate Roles

Element	Description
Primary actors	Project Owner
Main flow	The owner enters the project title, description, and technology stack, requests AI suggestions for setup and roles, reviews/edits the suggestions, saves and publishes the project, and reviews tracks without assigned members.
Alternative / exception flow	If AI generation fails, the owner can enter roles manually.

UC-05 — Manage Project Tasks

Element	Description
Primary actors	Team Member
Main flow	A team member opens the workspace and task board, creates and assigns a task, and the assignee receives an update. The assignee changes task status and the team sees the updated progress.
Alternative / exception flow	If the user is offline, the source proposes queueing the operation or asking the user to reconnect; the exact offline behavior is Needs Confirmation.

Diagram source note: the supplied file named “Dev Sync use case.svg” is a multi-page diagrams.net export (86 embedded diagram pages). Its rendered SVG page is titled “Full ERD (all entities)”; the supplied textual project analysis separately provides the five use cases above. The use-case information is reproduced here in text so that actors and flows remain readable and editable. The SVG is not misrepresented as a UML use-case diagram.



6.1 Related User Journeys

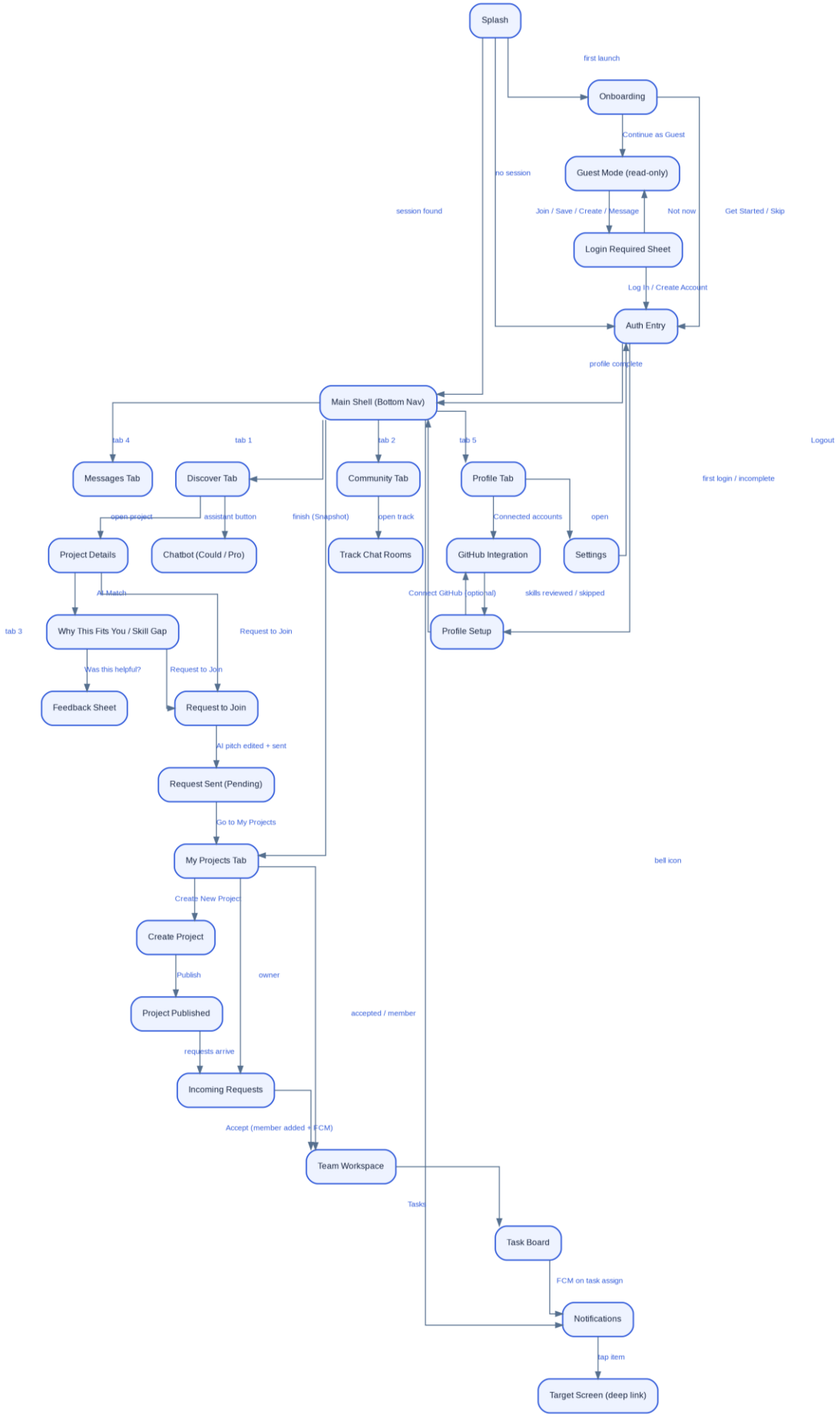
The project analysis also describes broader flows for authentication/profile setup, discovery, project application, project creation, missing-role discovery, workspace/tasks, communication, notifications/deep links, feedback, and plan-based AI access. These are workflow descriptions, not additional formal use cases in the five-use-case list above.

6.2 State Diagrams

This section translates the supplied diagrams.net state/lifecycle pages into embedded diagrams for the Word specification. The diagrams preserve the states and transition labels in the source file “Dev Sync State - Whole App Flow.drawio.svg”. They describe the overall application navigation flow and the individual lifecycles for join requests, projects, tasks, invitations, and GitHub-detected skills.

6.2.1 Whole Application Flow

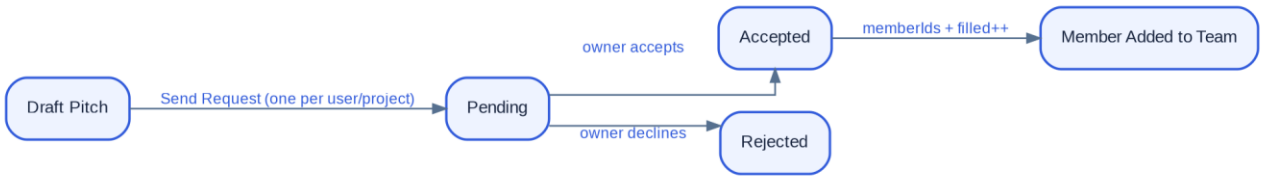
The overview starts at Splash and proceeds through Onboarding and the authentication entry. A user may enter the permitted Guest read-only mode or sign in and complete profile setup. The signed-in main shell exposes the primary tabs and routes to discovery, project details, join requests, project creation, team workspace, tasks, notifications/deep links, settings, and other features represented in the supplied flow.


Splash
first launch
Onboarding
Continue as Guest
no session
Guest Mode (read-only)
session found
Join / Save / Create / Message
Not now
Get Started / Skip
Login Required Sheet
Log In / Create Account
Auth Entry
profile complete
Main Shell (Bottom Nav)
tab 4
tab 1
tab 2
tab 5
Logout
Messages Tab
Discover Tab
Community Tab
Profile Tab
first login / incomplete
open project
assistant button
finish (Snapshot)
open track
Connected accounts
open
Project Details
Chatbot (Could / Pro)
Track Chat Rooms
GitHub Integration
Settings
AI Match
Connect GitHub (optional)
skills reviewed / skipped
tab 3
Why This Fits You / Skill Gap
Request to Join
Profile Setup
Was this helpful?
Request to Join
Feedback Sheet
Request to Join
AI pitch edited + sent
Request Sent (Pending)
Go to My Projects
My Projects Tab
bell icon
Create New Project
Create Project
Publish
owner
Project Published
accepted / member
requests arrive
Incoming Requests
Accept (member added + FCM)
Team Workspace
Tasks
Task Board
FCM on task assign
Notifications
tap item
Target Screen (deep link)

Source note: this overview reproduces the “00 - Whole App Flow” page. The diagram includes Community and Chatbot navigation nodes because they appear in the supplied flow source; their release status remains subject to the scope decisions recorded elsewhere in this document.

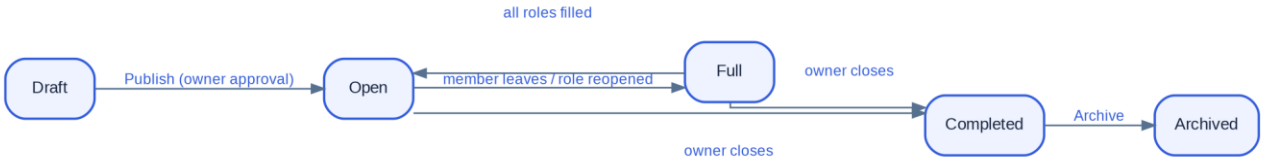
6.2.2 Join Request Lifecycle

A request is prepared as a Draft Pitch and becomes Pending when sent. The owner may accept or reject it. Acceptance adds the applicant to the team and increments the membership/filled count; rejection closes the request.



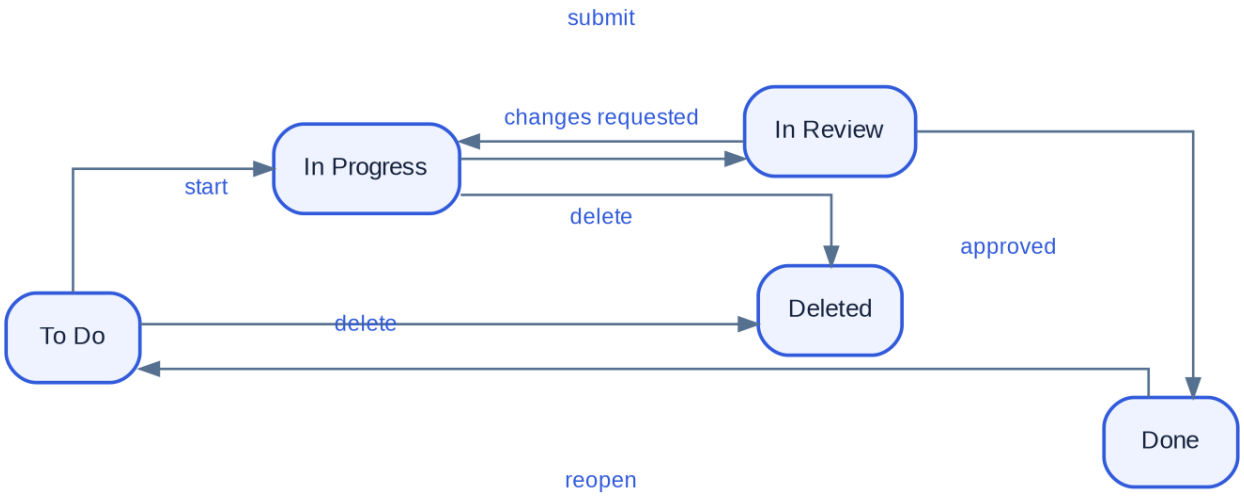
6.2.3 Project Lifecycle

A project moves from Draft to Open after owner-approved publication. It becomes Full when all roles are filled. A member leaving may reopen capacity; the owner may close an Open or Full project as Completed, then archive it.



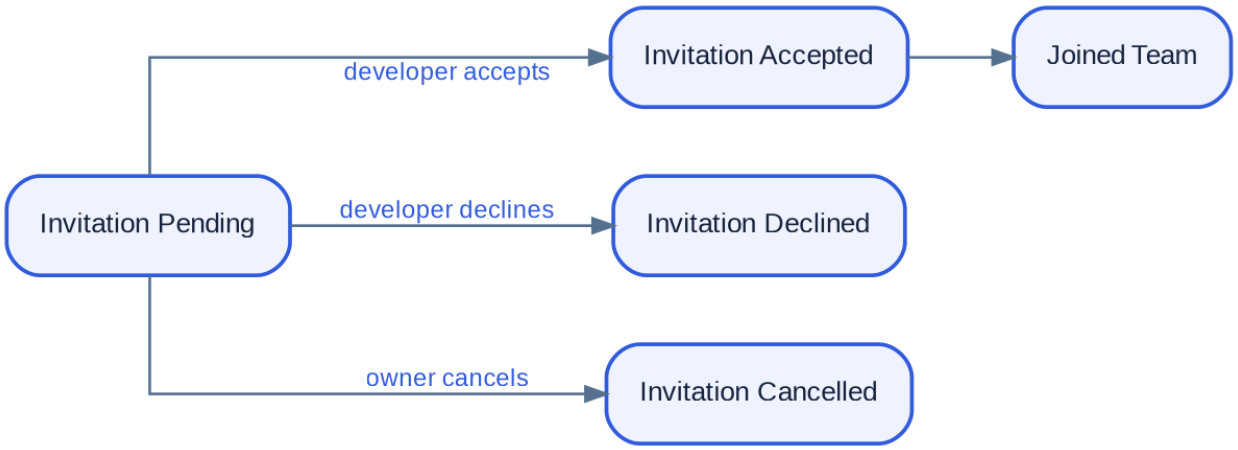
6.2.4 Task Lifecycle

Tasks progress from To Do to In Progress and then In Review. Review may return a task to In Progress when changes are requested, or move it to Done when approved. A completed task may be reopened to To Do; To Do and In Progress tasks may be deleted.



6.2.5 Invitation Lifecycle

An owner-created invitation begins as Invitation Pending. The developer may accept or decline, while the owner may cancel. Acceptance proceeds to Joined Team; the source diagram does not show further transitions from the terminal outcomes.



6.2.6 GitHub-Detected Skill Lifecycle

A skill inferred from GitHub enters Pending with confidence/evidence. The user may Add it or Dismiss it. An accepted skill is shown separately as AI-detected in the profile. A re-sync may surface a previously rejected skill again; disconnecting GitHub may remove the imported skill.



Diagram interpretation note: the source includes initial/final markers that are not text-labelled states. The embedded diagrams focus on named states and labelled transitions to keep the Word content readable. No additional states or transitions have been inferred.

7. Functional Requirements

The requirements below consolidate the functional requirement identifiers and module mapping in the supplied project analysis with the more detailed Firestore schema and feature descriptions. Priorities are retained where the source provides them. A requirement marked Needs Confirmation is not treated as committed implementation scope.

ID	Area	Functional requirement	Actor	Priority / status
FR-01	Authentication	The system shall allow a user to create an account and sign in using Google or Email through Firebase Authentication/FirebaseUI.	Developer	Must
FR-02	Guest access and session gating	The system shall allow Guests to browse publicly readable projects. When a Guest attempts an account-required action (such as creating a project or submitting a join request), the system shall require authentication and return the user to the original destination where supported.	Guest; System	Could / Needs Confirmation
FR-03	Developer profile	The system shall allow a signed-in user to create, view, and edit a developer profile containing the required name, track, level, and at least one programming language, with optional biography, skills, interests, education/background, links, and other supported profile fields. The system shall assign a unique public ID.	Developer	Must
FR-04	GitHub public data import	The system shall allow a user to enter a GitHub username and retrieve public repository summaries, repository languages/topics, language distribution, and last-activity information through the public GitHub REST API. The app shall use a single repository-list request where possible and throttle repeat synchronization using the stored lastSync timestamp (approximately one hour).	Developer; System	Must
FR-05	Detected skill review	The system shall present AI/GitHub-detected skills separately from manually confirmed skills and allow the user to accept or reject each detected skill before it is added to the profile.	Developer	Must

FR-06	Project discovery and filtering	The system shall display open projects and allow users to browse and filter them by supported criteria, including purpose, difficulty, required/open track, and technology stack. Newest results shall support pagination.	Developer; Guest	Must
FR-07	Project matching and ranking	The system shall rank available projects using the matching inputs described in the schema: skill overlap, whether the user's track is open, level relative to project difficulty, interests, and project purpose. The system shall store the resulting match data for the user.	System	Must
FR-08	Explainable match presentation	For each displayed AI/rule-based match, the system shall show an explanation and relevant skill context. A match score shall not be displayed alone; matched skills and missing skills shall be available with the result.	Developer; System	Must
FR-09	Skill-gap guidance	The system shall compare the user's available skills with project requirements, identify missing skills, and provide a short learning tip for each missing skill when such guidance is available.	Developer; System	Must
FR-10	AI output feedback	The system shall allow a user to mark an AI output as helpful or not helpful and optionally rate explanation clarity from 1 to 5. When the output is marked not helpful, the system may request a reason. Feedback shall be associated with the user, output kind, and prompt version.	Developer	Must
FR-11	Prompt evaluation	The system shall group feedback by prompt version and present evaluation information including rating count, helpful rate, and average clarity so that prompt versions can be compared.	Project team / evaluator	Must
FR-12	Project creation and editing	The system shall allow a project owner to create and edit a project draft with title, description, purpose, difficulty, owner track, one or more team tracks, capacity per track, and optional technology stack and project-specific details.	Project Owner	Must
FR-13	Project publication and capacity management	The system shall publish a project only after explicit owner confirmation. Before publication it shall validate required fields, at least one track, unique tracks, valid capacity, and ownerTrack membership. At publication, the owner shall fill one slot in ownerTrack and be counted in teamSize and filledCount. The project shall become full when all track slots are filled.	Project Owner; System	Must
FR-14	AI project setup and role suggestions	Where enabled, the system shall accept a short project idea and generate editable suggestions for project description, technology stack, roles/tracks, difficulty, and suggested tasks. The owner shall review and edit suggestions before saving or publishing. If AI generation fails, manual entry shall remain available.	Project Owner; System	Should
FR-15	Collaboration join requests	The system shall allow a signed-in developer to submit a join request for an open track in a project. The request shall include the selected track and an AI-drafted pitch reviewed/edited by the applicant. A composite project/applicant identity shall prevent duplicate requests. The owner shall be able to accept or reject a pending request, subject to track capacity.	Developer; Project Owner	Must
FR-16	Project invitations	Where enabled, the system shall allow an owner to invite a specific developer to a selected track. The invitee shall be able to accept or decline a pending invitation, and the owner shall be able to cancel a pending invitation. Acceptance shall fail with a clear message if the track has no remaining capacity or the invitee is already a member. A pending invitation does not reserve a slot.	Project Owner; Invitee	Should
FR-17	In-app notifications	The system shall create notification records for join requests, request decisions, invitations, and task assignment/status changes. The recipient shall see notifications through a Firestore stream, an unread indicator, and local notifications while the app is running. The Spark-only implementation shall not claim push delivery when the app is closed.	System; Recipient	Must (in-app); push is excluded
FR-18	Task management	The system shall allow team members to create tasks with a title, optional description, optional assignee, status, priority, and optional deadline. An assignee must be a project team member. Users shall be able to view task details and update task status; status changes use a picker rather than drag-and-drop.	Team Member	Must
FR-19	Team communication	If retained in the selected release scope, the system shall support individual and team messages associated with the relevant user/team context. Message storage is marked Should in the schema; Community and Chatbot are not included in the current schema baseline.	User; Team Member	Should / Needs Confirmation
FR-20	Project workspace and membership	The system shall provide team members with access to project workspace information, team members, and project tasks. Team data shall be readable only by project members. Creation of the team at project publication remains a proposed schema decision and requires approval.	Project Owner; Team Member	Should / Needs Confirmation
FR-21	Match history	Where enabled, the system shall retain a user's match results and display them ordered by creation time as Match History.	Developer	Could
FR-22	Saved projects and interests	Where enabled, the system shall let a user save/unsave a project and separately express interest. Express Interest shall not create a join request. Saving, expressing interest, and requesting to join shall remain distinct actions.	Developer	Could

FR-23	Free/Pro feature gating	The system shall provide Free and Pro Android build flavors and check whether a requested feature is enabled in the current flavor. If unavailable, the app shall show plan information without inventing prices, payment flows, usage limits, or benefits that have not been approved. Payment processing is out of scope.	User; System	Must
FR-24	Android interaction and animation	The system shall implement the agreed shared visual system and specified implicit/advanced animations using Flutter animation and transition capabilities, while maintaining consistent interaction patterns and accessible touch targets.	All users	Should for advanced animations; baseline UX Must
FR-25	Community feed and public developer discovery	Earlier planning describes Community feed, public developer profiles, and developer discovery. However, the Firestore schema changelog states that Community was removed from scope. This functionality shall be implemented only if the team confirms it is reinstated and defines its data/security model.	Developer; Guest	Needs Confirmation / excluded from current schema

8. Non-Functional Requirements

The following list retains all 14 NFR identifiers and measurable targets supplied by the project analysis. Where the source does not define a measurable target for a requested category, the gap is explicitly marked Needs Confirmation rather than assigned an invented threshold.

ID	Category	Requirement	Acceptance measure
NFR-01	Performance	The application shall load/display project recommendations within 5 seconds under the defined test conditions.	≤ 5 seconds
NFR-02	Performance / notifications	Notification delivery/visibility shall meet the source target on real devices. Under the Spark-only architecture this target applies to Firestore-stream/in-app and local-notification behavior while the app is running, not push delivery while closed.	≤ 10 seconds on real devices; scope adjusted to Spark capability
NFR-03	Security / authorization	Firestore Security Rules shall enforce least-privilege access so that users can read/write only the data permitted for their role and relationship to the project.	100% of the security checklist
NFR-04	Security / secrets	No API keys, private secrets, or credentials shall be embedded in the application source or repository. No secret may be placed in client-readable prompt templates.	No secrets found in repository review
NFR-05	Android security	Runtime permissions shall be requested only when needed and in accordance with the Android security model.	All permission requests reviewed and tested
NFR-06	Release security	Production Android release builds shall use code obfuscation.	AAB built with obfuscation enabled
NFR-07	Privacy	The app shall use only GitHub data needed for matching and shall provide a way for the user to disconnect the GitHub association. Public GitHub API access in the Spark plan shall not require a user token.	Disconnect option available and tested
NFR-08	Usability	Users in testing shall be able to complete the core tasks—sign in, find a project, and submit a request—without assistance.	Core tasks completed without moderator help
NFR-09	Usability / consistency	The application shall use a shared theme and widget kit, maintain consistent layouts, and provide smooth transitions/animations consistent with the design system.	Shared theme/widget kit used; visual and interaction review
NFR-10	Reliability	The application shall handle network failures with clear error messages and maintain stable sessions.	At least 95% crash-free sessions
NFR-11	Reliability / AI fallback	If the LLM is slow or unavailable, the app shall avoid a blank results screen and show cached or rule-based alternatives with an explanatory notice where available.	Fallback result shown instead of blank screen
NFR-12	Compatibility	The Android application shall support current Android devices and different screen sizes.	Tested on at least two real devices
NFR-13	Maintainability / engineering process	The project shall follow the agreed layered architecture, Git branching strategy, and pull-request review process. No direct push to main or develop; each merge requires review.	100% of merges reviewed
NFR-14	Documentation	Technical documentation and README shall be complete by the final milestone.	Completed by M5

8.1 Additional Quality Categories Requested

Category	Source-supported statement	Status
Scalability	The source describes a small demo dataset (approximately 15–20 projects) and Firestore query/index patterns, but does not define growth, concurrency, or load targets.	Needs Confirmation
Data integrity	Profile public IDs shall be unique; project publication and request/invitation acceptance shall update related records consistently; track filled counts shall never exceed capacity; duplicate requests/invitations shall be prevented by composite IDs.	Specified as transaction/business rules; test all listed cases
Availability	No uptime percentage or service availability target is defined. The Spark plan relies on Firebase and client-side service calls.	Needs Confirmation
Accessibility	The design system specifies minimum 48×48 px touch targets and clear typography, but no formal WCAG or accessibility conformance target is defined.	Needs Confirmation

8.2 Data Integrity and Business Rules

- Profile creation shall generate a candidate publicId, check uniqueness, and retry if the identifier already exists.
- Project publication shall validate title and description, at least one track, no duplicate tracks, count ≥ 1, and ownerTrack membership.
- The owner fills one slot in ownerTrack at publication; teamSize and filledCount include the owner.
- Track filled values shall never exceed count. openTrackKeys shall reflect tracks where filled < count.
- Accepting a join request shall require a pending request and an available slot; acceptance adds the applicant to team membership and updates track/project counts.
- Accepting an invitation shall require a pending invitation, a non-member invitee, and an available slot. A matching pending join request for the same project is also resolved as accepted.
- Guests may read projects only. Team data shall not be readable by non-members. Other profile access rules for Guests remain Needs Confirmation.
- Project publication and all AI-generated content require explicit user confirmation; no automatic publication is permitted.

9. Technical Architecture and Constraints

9.1 Confirmed Spark-Only Architecture

The latest supplied Firestore schema (version 1.1, dated 29 October 2026) explicitly records the decision to use Firebase Spark only. It states that the project does not use Cloud Functions or Firebase Storage. This supersedes older planning text that describes Cloud Functions as the server-side path for GitHub and LLM calls.

Concern	Current source-aligned implementation
Backend plan	Firebase Spark only; no Cloud Functions.
Authentication	Firebase Authentication with Google and Email providers.
Database	Cloud Firestore with Security Rules, indexes, and client-side transactions.
GitHub	Public GitHub API called from the app using githubUsername, without a token; approximately 60 unauthenticated requests/hour; throttle synchronization.
AI	Gemini through Firebase AI Logic; prompt templates read from Firestore. Templates are client-readable and must not contain secrets.
Notifications	Notification documents written by the app and read through Firestore streams; local notification while app runs; no push while app is closed.
File storage	No Firebase Storage. Photo and CV URLs point to an external file host; deletion/authorization limitations remain.
Usage limit	Daily AI usage counter is a soft limit checked in the app; exact limit is not set.
Transactions	Project publish, request/invitation acceptance, and related updates are performed in the app.

9.2 Security and Privacy Considerations

- Apply least-privilege Firestore rules to user, project, request, invitation, team, task, feedback, prompt, usage, and notification paths.
- Keep private information such as email and tokens out of public user profile documents.
- Do not store GitHub access tokens in the Spark-only implementation; public repository data is read without authentication.
- Prompt templates are readable by signed-in users; never place credentials or secrets in prompts.
- CV and photo files are hosted externally. The source notes that anyone with a CV link may open it and that replaced/deleted files may remain on the host. The UI should restrict who sees the CV link, but the file itself is not protected by Firestore rules.
- Account deletion is proposed as an app-side batch operation. Files remaining on the external file host are a known limitation and require acceptance/confirmation.

10. Design System Summary

Design area	Source specification
Brand	Blue is the sole brand signature color; purple/violet are prohibited.
Light primary	#315BDB; pressed #2548B8; dark blue accent #173B9A.
Dark primary	#6F91FF; pressed #8AA5FF; dark blue accent #315BDB.
Typography	Inter; defined headline, title, body, and label scales.
Spacing	8-point grid; 16 px screen margins; 12–16 px card/item gutters; 24 px group spacing.
Touch targets	Minimum 48×48 px.
Shapes	12 px buttons and inputs; 16 px cards; full pill radius for skill/status chips.
Elevation	Flat tonal layering and thin borders; no default diffuse drop shadows.
Navigation	Five tabs: Discover, Community, My Projects, Messages, Profile; notification bell in top app bar.
Themes	Light and dark modes supported; light is the default in the project analysis.

11. Verification and Acceptance Summary

Verification area	Acceptance evidence
Core user journey	Demonstrate profile completion, discovery/matching, request submission, owner decision, team membership, and task workflow end-to-end.
Matching quality	At least 60% of test recommendations rated suitable; average explanation clarity at least 4/5.
Performance	Recommendation response time ≤ 5 seconds.
Notifications	In-app/local notification behavior ≤ 10 seconds on real devices, subject to the Spark-only limitation.
Reliability	At least 95% crash-free sessions; clear network error handling and AI fallback.
Security	100% completion of the Firestore rules/security checklist; no secrets in repository; runtime permission review; obfuscated release AAB.
Compatibility	Test on at least two real Android devices and different screen sizes.
Engineering process	100% of merges reviewed; use main/develop/feature branches as specified.
Delivery	All Must features complete for M4; technical documentation and README complete by M5.

12. Source Basis and Document Notes

This document is based on the supplied DevSync comprehensive project analysis, the DevSync Firestore schema and implementation notes (including Spark-only decision and v1.1 changelog), the DevSync Mobile design-system specification, and the supplied multi-page diagrams.net SVG. The textual use cases UC-01 to UC-05 are reproduced from the project analysis.

The requirements consolidate the source's requirement IDs and module mapping with detailed behaviors from the schema and feature descriptions. Where the source contains conflicting or undecided material, this document labels it Needs Confirmation. No Cloud Functions, Firebase Storage, payment processing, iOS release, or unsupported feature has been introduced as current committed scope.

Document status: source-aligned draft for team review. Items explicitly marked Needs Confirmation should be resolved by the project team before they are treated as approved requirements.